SciPy Notes

Introduction:

What is SciPy?

SciPy (Scientific Python) is a powerful library for scientific computing in Python. **SciPy stands for "Scientific Python."** It builds on **NumPy** and provides additional functionalities like:

- Optimization
- Integration
- Interpolation
- Signal processing
- Linear algebra
- Statistics

SciPy was created by **Travis Oliphant**, who also created NumPy.

Why Use SciPy with Pandas?

Pandas is used for handling and analyzing structured data (tables, data frames, CSV files), while SciPy provides advanced mathematical tools that can be applied to the data in Pandas.

NumPy can compute basic statistics like the mean, but SciPy provides advanced statistical tools like **hypothesis testing** and **probability distributions**.

What is SciPy Written In?

SciPy is mostly written in **Python**, which makes it easy to use. However, some parts of it are written in **C** to make calculations faster.

Installing SciPy and Pandas:

Before using SciPy, install them using pip:

- **pip install scipy pandas**

Checking SciPy Version

The version string is stored under the __version__ attribute.

- **import scipy**
- **print(scipy.__version__)**

Using SciPy for Statistical Analysis in Pandas:

Example Dataset

Let's create a simple dataset using Pandas:

```
data = {
   'Name': ['Alice',  'Bob', 'Charlie', 'David', 'Eva'],
   'Age':  [23, 25, 24, 22, 26],
   'Score':  [85, 90, 88, 75, 95]
}

df = pd.DataFrame(data)
print(df)
```

Calculating Basic Statistics

We can use scipy.stats to compute statistics on a Pandas DataFrame.

Mean, Median, and Mode

```
mean_score = df['Score'].mean()  # Pandas method
median_score = df['Score'].median()  # Pandas method
mode_score = stats.mode(df['Score'])  # SciPy  method

print(f"Mean: {mean_score}, Median: {median_score}, Mode: {mode_score.mode[0]}")
```

Standard Deviation and Variance

```
std_dev = np.std(df['Score'])  # NumPy method
variance = np.var(df['Score'])  # NumPy method

print(f"Standard Deviation: {std_dev}, Variance: {variance}")
```

Hypothesis Testing with SciPy:

SciPy makes hypothesis testing easy, such as checking if two groups have significantly different means.

T-test (Comparing Two Groups)

Example: Comparing two different sets of scores.

```
group1 = [85, 90, 88, 75, 95]
```

```
group2 = [78, 82, 80, 76, 88]
t_statistic, p_value = stats.ttest_ind(group1, group2)
print(f"T-Statistic: {t_statistic}, P-Value: {p_value}")
```

- If **p-value < 0.05**, we reject the null hypothesis (meaning the two groups are significantly different).

Data Distribution with SciPy:

SciPy helps us understand how data is distributed.

Normal Distribution (Gaussian Distribution)

Let's generate random data and visualize its distribution.

```
import matplotlib.pyplot as plt
data = np.random.normal(loc=50, scale=10, size=1000)  # Mean=50, Std=10, Size=1000
plt.hist(data, bins=30, alpha=0.5, color='blue', edgecolor='black')
plt.title("Normal Distribution")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
```

Skewness and Kurtosis

- **Skewness** tells us if the data is symmetric.
- **Kurtosis** tells us how sharp the peak is.

```
skewness = stats.skew(data)
kurtosis = stats.kurtosis(data)
print(f"Skewness: {skewness}, Kurtosis: {kurtosis}")
```

Linear Regression with SciPy:

Linear regression helps us find relationships between variables.

Example: Predicting Scores Based on Age

```
ages = np.array(df['Age'])
scores = np.array(df['Score'])
slope, intercept, r_value, p_value, std_err = stats.linregress(ages,  scores)
print(f"Slope: {slope}, Intercept: {intercept}")
```

We can now predict a new score:

```
predicted_score = slope * 27 + intercept
print(f"Predicted Score for Age 27: {predicted_score}")
```

Interpolation Using SciPy:

Interpolation helps estimate missing data points.

```
from scipy.interpolate import interp1d

x = np.array([1, 2, 3, 4, 5])
y = np.array([10, 20, 30, 40, 50])

interpolation = interp1d(x, y, kind='linear')

# Predict value at x = 2.5
predicted_value = interpolation(2.5)
print(f"Predicted value at x=2.5: {predicted_value}")
```

Optimizing Data with SciPy:

SciPy allows us to find optimal values for functions.

Example: Minimizing a Function

```
from scipy.optimize import minimize

def function_to_minimize(x):
    return x**2 + 5*x + 6  # Example function

result = minimize(function_to_minimize, x0=0)  # x0 is the starting guess
print(f"Optimal value: {result.x}")
```

SciPy Constants:

1. Understanding Scientific Constants

Scientific constants are predefined numerical values that are universally accepted in physics and mathematics.
For example, **π (pi)** is a mathematical constant used in trigonometry and geometry.

Example: Getting the value of **π (pi)** in SciPy

```
from scipy import constants

print(constants.pi)  # Output: 3.141592653589793
```

2. Listing All Available Constants

SciPy provides a large number of constants categorized into different unit types. We can list all available constants using the dir() function.

Example:

```
from scipy import constants

print(dir(constants))  # Lists all constants in the module
```

3. Unit Categories in SciPy Constants

The constants module provides values categorized into various **SI (International System of Units) and Non-SI** units.

Main Unit Categories:

1. **Metric (SI) Prefixes**
2. **Binary Prefixes**
3. **Mass**
4. **Angle**
5. **Time**
6. **Length**
7. **Pressure**
8. **Volume**
9. **Speed**
10. **Temperature**
11. **Energy**
12. **Power**
13. **Force**

4. Metric (SI) Prefixes

Metric prefixes are used to indicate multiples or fractions of a unit in **powers of 10**.

Prefix	Symbol	Value
Yotta	Y	102410^{24}1024
Zetta	Z	102110^{21}1021
Exa	E	101810^{18}1018
Peta	P	101510^{15}1015
Tera	T	101210^{12}1012
Giga	G	10910^9109
Mega	M	10610^6106
Kilo	k	10310^3103
Hecto	h	10210^2102
Deka	da	10110^1101

Deci	d	10−110^{-1}10−1
Centi	c	10−210^{-2}10−2
Milli	m	10−310^{-3}10−3
Micro	µ	10−610^{-6}10−6
Nano	n	10−910^{-9}10−9
Pico	p	10−1210^{-12}10−12
Femto	f	10−1510^{-15}10−15
Atto	a	10−1810^{-18}10−18
Zepto	z	10−2110^{-21}10−21

Example: Getting values of metric prefixes in SciPy

```
from scipy import constants
print(constants.kilo) # 1000.0
print(constants.milli) # 0.001
print(constants.micro) # 1e-6
```

5. Binary Prefixes

Binary prefixes are used to represent units in terms of **bytes (data storage)**.

Prefix	Symbol	Value (Bytes)
Kibi	Ki	1024
Mebi	Mi	1024^2
Gibi	Gi	1024^3
Tebi	Ti	1024^4
Pebi	Pi	1024^5
Exbi	Ei	1024^6
Zebi	Zi	1024^7
Yobi	Yi	1024^8

Example: Getting values of binary prefixes

```
from scipy import constants
print(constants.kibi) # 1024
print(constants.mebi) # 1048576
print(constants.gibi) # 1073741824
```

6. Mass

The mass constants return their values in **kilograms (kg)**.

Unit	Symbol	Value (kg)
Gram	g	0.001
Metric ton	t	1000.0
Pound	lb	0.453592
Ounce	oz	0.0283495
Stone	st	6.35029

Example:

```
from scipy import constants
print(constants.gram)  # 0.001
print(constants.pound) # 0.453592
print(constants.ounce) # 0.0283495
```

7. Angle

Angle values are represented in **radians**.

Unit	Symbol	Value (radians)
Degree	°	0.01745
Arcminute	'	0.00029089
Arcsecond	"	4.8481e-06

Example:

```
from scipy import constants
print(constants.degree)  # 0.017453292519943295
print(constants.arcsecond) # 4.84813681109536e-06
```

8. Time

Time values are in **seconds (s)**.

Unit	Symbol	Value (s)
Minute	min	60
Hour	hr	3600
Day	d	86400
Week	wk	604800

Example:

```
from scipy import constants
print(constants.hour) # 3600.0
print(constants.day)  # 86400.0
```

9. Length

Length values are in **meters (m)**.

Unit	Value (m)
Inch	0.0254
Foot	0.3048
Yard	0.9144
Mile	1609.34
Nautical mile	1852.0

Example:

```
from scipy import constants
print(constants.inch) # 0.0254
print(constants.mile) # 1609.34
```

10. Speed

Speed is given in **meters per second (m/s)**.

Unit	Value (m/s)
km/h	0.2778
Mph	0.44704
Speed of sound	340.5

Example:

```
from scipy import constants
print(constants.mph)  # 0.44704
print(constants.speed_of_sound) # 340.5
```

11. Temperature

Temperature is given in **Kelvin (K)**.

Unit	Value (K)
0°C (Celsius)	273.15
1°F (Fahrenheit)	0.5556

Example:

```
from scipy import constants
print(constants.zero_Celsius)  # 273.15
```

12. Energy

Energy is given in **Joules (J)**.

Unit	Value (J)
Electron volt	1.6×10−191.6 \times 10^{-19}1.6×10−19
Calorie	4.184

Example:

```
from scipy import constants
print(constants.calorie)  # 4.184
```

SciPy Optimizers:

What are Optimizers?

Optimizers in SciPy help us either:

1. **Find the minimum value** of a function (like the lowest point on a curve).
2. **Find the root** of an equation (the point where the function equals zero).

Finding the Root of an Equation

A **root** is the value of xxx where a function becomes zero.

Why Use SciPy Instead of NumPy?

NumPy can find roots for simple equations like polynomials but struggles with **non-linear equations** (which are more complex).

For example, let's find the root of this equation:

- x+cos(x)=0

We can use `scipy.optimize.root()` to solve it.

Code Example: Finding the Root

```
from scipy.optimize import root
from math import cos

# Define the equation
def eqn(x):
  return x + cos(x)

# Find the root starting with an initial guess of x=0
myroot = root(eqn, 0)

# Print the solution (root value)
print(myroot.x)
```

Explanation:

1. We define a function `eqn(x)` representing x+cos⁡(x)x + \cos(x)x+cos(x).
2. We use `root(eqn, 0)` to find the root, starting with an initial guess of x=0x = 0x=0.
3. The result contains a lot of information, but the actual root is stored in `myroot.x`.

If you want to see all details about the solution, print the full object:

```
print(myroot)
```

Finding the Minimum of a Function

A function's graph may have **high points (maxima)** and **low points (minima)**.

- **Global Minimum**: The lowest point in the entire curve.
- **Local Minimum**: A low point in a certain region, but not the lowest overall.

To find the **minimum** of a function, we use scipy.optimize.minimize().

Code Example: Finding the Minimum

Let's minimize this function:

f(x)=x2+x+2f(x) = x^2 + x + 2f(x)=x2+x+2

```
from scipy.optimize import minimize
# Define the function
def eqn(x):
  return x**2 + x + 2
# Find the minimum using the 'BFGS' method, starting from x=0
mymin = minimize(eqn, 0, method='BFGS')

# Print the result
print(mymin)
```

Explanation:

1. The function x2+x+2x^2 + x + 2x2+x+2 is a **parabola** that opens upwards, meaning it has a minimum.
2. We use minimize(eqn, 0, method='BFGS') to find this minimum.
    - **eqn** is the function to minimize.
    - **0** is our starting guess.
    - **BFGS** is the optimization method.
3. The result gives details about the minimum value and where it occurs.

You can also customize the behavior using extra options like:

```
mymin = minimize(eqn, 0, method='BFGS', options={"disp": True})
```

- "disp": True makes SciPy print extra details about the process.

SciPy Sparse Data:

What is Sparse Data?

Sparse data means having a dataset where most of the values are **zero** or **empty**. For example:

[1, 0, 2, 0, 0, 3, 0, 0, 0, 0, 0, 0]

In this example, most of the numbers are **0**, which means it is a **sparse array**.

- **Sparse Data** → Most values are **zero** (unused)
- **Dense Data** → Most values are **non-zero** (used)

Sparse data is commonly found in scientific computing, machine learning, and linear algebra, where handling such data efficiently is essential.

Why Use Sparse Matrices?

Handling sparse data as a normal array takes up **a lot of memory**, especially if it's large. SciPy provides **sparse matrices** to store only the non-zero values, making computations **faster** and **more efficient**.

How to Work With Sparse Data in SciPy?

SciPy has a module called scipy.sparse that helps us manage sparse data efficiently.

There are two important types of **sparse matrices**:

1. **CSC (Compressed Sparse Column)** → Best for fast **column slicing**.
2. **CSR (Compressed Sparse Row)** → Best for fast **row slicing** and matrix operations.

In this guide, we'll focus on **CSR matrices**.

Creating a CSR Matrix

We can create a CSR matrix using the csr_matrix() function.

Example:

```
import numpy as np
from scipy.sparse import csr_matrix
# Creating a NumPy array
arr = np.array([0, 0, 0, 0, 0, 1, 1, 0, 2])
# Converting to a CSR sparse matrix
sparse_matrix = csr_matrix(arr)
print(sparse_matrix)
```

Output:

```
(0, 5)	1
(0, 6)	1
(0, 8)	2
```

This means:

- A **1** is stored at row **0**, column **5**.
- A **1** is stored at row **0**, column **6**.
- A **2** is stored at row **0**, column **8**.

This way, only the **non-zero** values are stored, making it memory efficient.

Sparse Matrix Operations

Once we have a **CSR matrix**, we can perform different operations on it.

1. Viewing Non-Zero Elements

We can view the **stored (non-zero) values** using .data property.

```
import numpy as np
from scipy.sparse import csr_matrix
arr = np.array([[0, 0, 0], [0, 0, 1], [1, 0, 2]])
sparse_matrix = csr_matrix(arr)
print(sparse_matrix.data)
```

Output:

```
[1 1 2]
```

This tells us that the matrix contains **three** non-zero values: 1, 1, 2.

2. Counting Non-Zero Elements

We can count how many **non-zero** values exist using .count_nonzero().

```
print(sparse_matrix.count_nonzero())
```

Output:

```
3
```

This means there are **3 non-zero values** in the matrix.

3. Removing Zero Entries

If we want to **remove unnecessary zeros**, we can use .eliminate_zeros().

```
sparse_matrix.eliminate_zeros()
print(sparse_matrix)
```

This **removes zero placeholders** and optimizes the matrix further.

4. Summing Duplicate Entries

If there are duplicate values at the same position, we can **add them together** using .sum_duplicates().

```
sparse_matrix.sum_duplicates()
print(sparse_matrix)
```

This ensures that all values in the matrix are unique.

5. Converting CSR to CSC

We can convert a **CSR matrix** into a **CSC matrix** using .tocsc(). This is useful if column-wise operations are required.

```
csc_matrix = sparse_matrix.tocsc()
print(csc_matrix)
```

Final Thoughts

- Sparse matrices **save memory** by storing only the necessary data.
- **CSR matrices** are efficient for row-wise operations.
- **CSC matrices** are efficient for column-wise operations.
- SciPy provides many functions to **manipulate and optimize** sparse data.

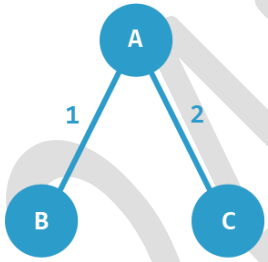
SciPy Graphs :

Graphs are a way to represent relationships between different elements (nodes). SciPy provides the scipy.sparse.csgraph module to work with graphs efficiently.

Adjacency Matrix

An **Adjacency Matrix** is a square table that shows how nodes (elements) in a graph are connected. If there is a connection, we put a number (called weight) in the matrix. If there's no connection, we put **0**.

Example of an Adjacency Matrix



Imagine a graph with three elements **A, B, and C**, where:

- A is connected to B with weight **1**.
- A is connected to C with weight **2**.
- C and B are **not connected**.

This can be written as:

	A	**B**	**C**
A	0	1	2
B	1	0	0
C	2	0	0

This table is called an adjacency matrix.

Working with Graphs in SciPy

1. Finding Connected Components

A **connected component** is a group of elements that are connected to each other.

Example: Find connected components

```
import numpy as np
from scipy.sparse.csgraph import connected_components
from scipy.sparse import csr_matrix
arr = np.array([
  [0, 1, 2],
  [1, 0, 0],
  [2, 0, 0]])
graph = csr_matrix(arr)
print(connected_components(graph))
```

2. Finding Shortest Paths

To find the shortest path between elements, we can use different algorithms:

Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest path from **one element** to others.

```
import numpy as np
from scipy.sparse.csgraph import dijkstra
from scipy.sparse import csr_matrix
arr = np.array([
  [0, 1, 2],
  [1, 0, 0],
  [2, 0, 0]])
graph = csr_matrix(arr)
print(dijkstra(graph, return_predecessors=True, indices=0))
```

- indices=0 means we are finding paths **starting from node 0**.
- return_predecessors=True gives the path taken.

Floyd Warshall Algorithm

Floyd-Warshall finds **the shortest path between all pairs** of elements.

```
import numpy as np
from scipy.sparse.csgraph import floyd_warshall
from scipy.sparse import csr_matrix
arr = np.array([
  [0, 1, 2],
  [1, 0, 0],
  [2, 0, 0]])
graph = csr_matrix(arr)
print(floyd_warshall(graph, return_predecessors=True))
```

Bellman-Ford Algorithm

Bellman-Ford is similar to Dijkstra but can handle **negative weights**.

```
import numpy as np
from scipy.sparse.csgraph import bellman_ford
from scipy.sparse import csr_matrix
arr = np.array([
  [0, -1, 2],
  [1, 0, 0],
  [2, 0, 0]
])

graph = csr_matrix(arr)

print(bellman_ford(graph, return_predecessors=True, indices=0))
```

This is useful when **some connections have a negative impact**, such as in financial graphs.

3. Traversing a Graph

Graph traversal means **visiting all elements** in a specific order.

Depth-First Search (DFS)

DFS visits as deep as possible before backtracking.

```
import numpy as np
from scipy.sparse.csgraph import depth_first_order
from scipy.sparse import csr_matrix
arr = np.array([
  [0, 1, 0, 1],
  [1, 1, 1, 1],
  [2, 1, 1, 0],
  [0, 1, 0, 1]
])
graph = csr_matrix(arr)
print(depth_first_order(graph, 1))
```

- 1 means starting from element **1**.

Breadth-First Search (BFS)

BFS visits elements **level by level** before moving deeper.

```
import numpy as np
from scipy.sparse.csgraph import breadth_first_order
from scipy.sparse import csr_matrix
arr = np.array([
  [0, 1, 0, 1],
  [1, 1, 1, 1],
  [2, 1, 1, 0],
  [0, 1, 0, 1]
])
graph = csr_matrix(arr)
print(breadth_first_order(graph, 1))
```

- This explores **all directly connected elements first**.

Summary

- **Adjacency Matrix**: Represents connections between elements in a graph.
- **Dijkstra**: Finds shortest path from **one element**.
- **Floyd-Warshall**: Finds shortest path between **all elements**.
- **Bellman-Ford**: Works with **negative weights**.
- **DFS (Depth First Search)**: Goes as deep as possible before backtracking.
- **BFS (Breadth First Search)**: Explores level by level.

SciPy Spatial Data:

What is Spatial Data?

Spatial data refers to data that represents points, shapes, or objects in a geometric space, like a map or a coordinate system.

Why Do We Use Spatial Data?

- To check if a point is inside a shape (e.g., is a GPS location inside a country border?).
- To find the nearest point to another point (e.g., nearest hospital to a user).
- To measure distances between points (e.g., how far two cities are from each other).

How Does SciPy Help?

SciPy provides the `scipy.spatial` module, which has useful tools to work with spatial data.

1. Triangulation

What is Triangulation?

- It divides a polygon (a shape with multiple sides) into **triangles**.
- This is useful for **finding the area** of irregular shapes.

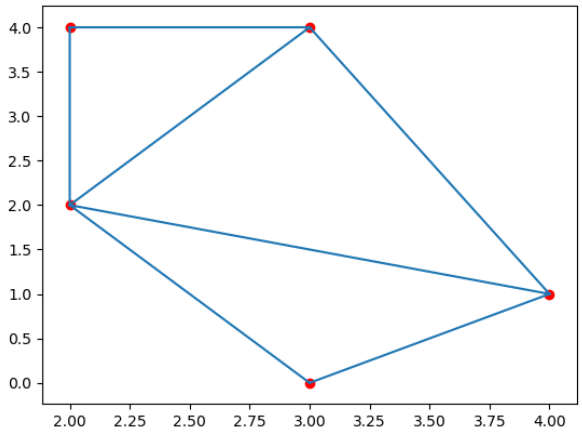
How to Do Triangulation in Python?

The `Delaunay()` function creates a set of triangles from given points.

Example:

```
import numpy as np
from scipy.spatial import Delaunay
import matplotlib.pyplot as plt
# Define 5 points in 2D space
points = np.array([
  [2, 4],  # Point 1
  [3, 4],  # Point 2
  [3, 0],  # Point 3
  [2, 2],  # Point 4
  [4, 1]   # Point 5
])
# Create triangles
simplices = Delaunay(points).simplices
# Plot the triangles
plt.triplot(points[:, 0], points[:, 1], simplices)
plt.scatter(points[:, 0], points[:, 1], color='r')  # Mark the points in red
plt.show()
```

This will create triangles connecting the given points.



2. Convex Hull

What is a Convex Hull?

- The **convex hull** is the **smallest shape that can enclose all given points** (like wrapping a rubber band around them).
- It is useful in **image processing, collision detection, and geographic mapping**.

How to Find a Convex Hull?

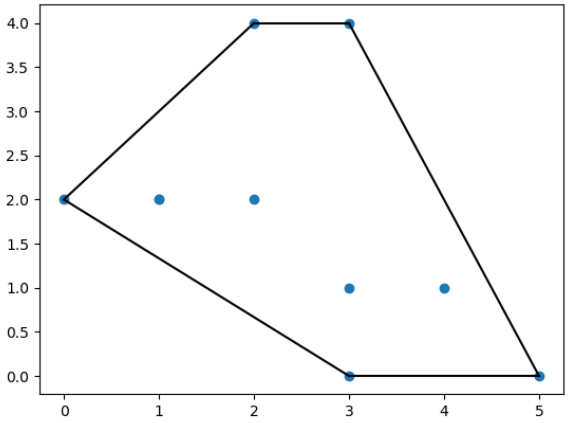
The ConvexHull() function helps us find the outer boundary of given points.

Example:

```
import numpy as np
from scipy.spatial import ConvexHull
import matplotlib.pyplot as plt
# Define 10 points
points = np.array([
  [2, 4], [3, 4], [3, 0], [2, 2], [4, 1],
  [1, 2], [5, 0], [3, 1], [1, 2], [0, 2]
])
# Compute the convex hull
hull = ConvexHull(points)
# Plot points and convex hull
plt.scatter(points[:, 0], points[:, 1])
for simplex in hull.simplices:
  plt.plot(points[simplex, 0], points[simplex, 1], 'k-')  # Black lines

plt.show()
```

This will plot the smallest polygon enclosing the points.



3. KD-Trees (Fast Nearest Neighbor Search)

What is a KD-Tree?

- A **KD-Tree** is a special data structure that quickly finds **the nearest neighbor** to a given point.
- It's used in **GPS systems, image recognition, and clustering algorithms**.

How to Find the Nearest Point?

The KDTree() function creates a tree from points, and query() finds the nearest one.

Example:

```
from scipy.spatial import KDTree
# Define some points
points = [(1, -1), (2, 3), (-2, 3), (2, -3)]

# Create a KD-Tree
kdtree = KDTree(points)
# Find the closest point to (1,1)
res = kdtree.query((1, 1))
print(res)
```

Output: (2.0, 0) means:

- The closest point is at index 0 (first point in the list).
- The **distance** to that point is 2.0.

4. Distance Metrics (Measuring Distance Between Points)

There are different ways to measure the distance between two points:

A) Euclidean Distance (Straight-Line Distance)

The shortest distance between two points (like using a ruler).

Example:

```
from scipy.spatial.distance import euclidean
p1 = (1, 0)
p2 = (10, 2)
res = euclidean(p1, p2)
print(res)  # Output: 9.22
```

This finds the straight-line distance between the two points.

B) City Block Distance (Manhattan Distance)

- Measures distance **only in horizontal and vertical directions** (like city streets).

Example:

```
from scipy.spatial.distance import cityblock
p1 = (1, 0)
p2 = (10, 2)
res = cityblock(p1, p2)
print(res)  # Output: 11
```

C) Cosine Distance (Angle Between Vectors)

- Measures how **similar two points are in direction**, rather than distance.
- Used in **text similarity and recommendation systems**.

Example:

```
from scipy.spatial.distance import cosine
p1 = (1, 0)
p2 = (10, 2)
res = cosine(p1, p2)
print(res)  # Output: 0.019
```

A smaller value means the points are **more similar** in direction.

D) Hamming Distance (For Binary Data)

- Measures **how different two binary sequences are**.
- Used in **error detection and DNA analysis**.

Example:

```
from scipy.spatial.distance import hamming
p1 = (True, False, True)
p2 = (False, True, True)
res = hamming(p1, p2)
print(res)  # Output: 0.67
```

67% of the bits are different between the two sequences.

Summary

Concept	What It Does	Where It's Used
Triangulation	Splits a shape into triangles	Mapping, 3D Graphics
Convex Hull	Smallest shape covering points	Collision Detection, Image Processing
KD-Tree	Finds the nearest point	GPS, Machine Learning
Euclidean Distance	Straight-line distance	Geometry, Physics
Cityblock Distance	Distance using only up/down/left/right moves	Traffic Systems
Cosine Distance	Measures angle similarity	Text Analysis, AI
Hamming Distance	Measures bit differences	DNA Analysis, Error Checking

SciPy Interpolation:

Interpolation is a method used to **find missing values** between given data points. It helps **smooth data** and is useful in **machine learning** for filling in missing values (also called **imputation**).

SciPy provides the scipy.interpolate module, which has several functions for **different types of interpolation**.

1D Interpolation (Simple Linear Interpolation)

This method estimates new values **between two known points** using straight lines.

How does it work?

- We have two lists: xs (x-values) and ys (y-values).
- The function interp1d() creates a **formula** that estimates new y values for any given x.
- We can then **call this function** with new x-values to get their estimated y-values.

Example: Interpolating values for x = 2.1, 2.2, ... 2.9

```
from scipy.interpolate import interp1d
import numpy as np
# Define known data points
xs = np.arange(10)      # x-values: 0,1,2,...9
ys = 2*xs + 1           # y-values: following the equation y = 2x + 1
# Create interpolation function
interp_func = interp1d(xs, ys)
# Find y-values for new x-values
newarr = interp_func(np.arange(2.1, 3, 0.1))
print(newarr)
```

Output:

```
[5.2  5.4  5.6  5.8  6.   6.2  6.4  6.6  6.8]
```

This means that for x = 2.1, the interpolated y value is **5.2**, for x = 2.2 it is **5.4**, and so on.

Important Note:
You cannot use interp_func() for values **outside** the original x-range (0 to 9), or it will give an error.

Spline Interpolation (Smooth Curves Instead of Straight Lines)

- Unlike interp1d(), which creates a **straight line** between points, UnivariateSpline() fits a **smooth curve** using small polynomial pieces (**splines**).
- This method is useful when data is **non-linear** (not a straight line).

Example: Finding smooth interpolated values

```
from scipy.interpolate import UnivariateSpline
import numpy as np
# Define known data points
xs = np.arange(10)          # x-values: 0,1,2,...9
ys = xs**2 + np.sin(xs) + 1    # y-values: nonlinear function
# Create a smooth interpolation function
interp_func = UnivariateSpline(xs, ys)
# Find new values
newarr = interp_func(np.arange(2.1, 3, 0.1))
print(newarr)
```

Output:

```
[5.628  6.039  6.471  6.922  7.393  7.885  8.396  8.927  9.479]
```

This method gives **smooth** estimates instead of just straight lines.

Radial Basis Function (More Flexible Interpolation)

- This method is useful when data points are scattered in **irregular patterns**.
- Rbf() creates an **approximate function** based on distances from known points.

Example: Interpolating with RBF

```
from scipy.interpolate import Rbf
import numpy as np
# Define known data points
xs = np.arange(10)          # x-values
ys = xs**2 + np.sin(xs) + 1    # y-values: nonlinear function
# Create an interpolation function using RBF
interp_func = Rbf(xs, ys)
# Find new values
newarr = interp_func(np.arange(2.1, 3, 0.1))
print(newarr)
```

Output:

```
[6.257  6.621  7.003  7.401  7.816  8.247  8.695  9.160  9.642]
```

RBF interpolation works well when the data is not evenly spaced or follows complex patterns.

Summary

Interpolation helps us estimate missing values.
Three methods in SciPy:

1. **Linear interpolation (`interp1d`)** → Uses **straight lines**.
2. **Spline interpolation (`UnivariateSpline`)** → Uses **smooth curves**.
3. **Radial Basis Function (`Rbf`)** → Works with **irregular data**.

When to use what?

✓ Use `interp1d()` for **simple cases** with straight lines.

✓ Use `UnivariateSpline()` for **smooth curves**.

✓ Use `Rbf()` when data is **irregular or complex**.

SciPy Statistical Significance Tests:

Statistical significance tests help us determine if a result happened due to chance or if there is a real reason behind it. SciPy provides various functions to perform these tests, making it easier to analyze data.

Key Concepts in Statistics

1. Hypothesis in Statistics

A hypothesis is an assumption about something in a population.

- **Null Hypothesis (H_0)**: Assumes that there is no significant difference or effect.
- **Alternate Hypothesis (H_1)**: Assumes that there is a significant difference or effect.

Example:

- Null Hypothesis: *"The student is not better than average."*
- Alternate Hypothesis: *"The student is better than average."*

2. One-Tailed vs. Two-Tailed Tests

- **One-tailed test**: Checks if a value is either *greater than* or *less than* a given value.
- **Two-tailed test**: Checks if a value is *different* (could be either greater or smaller).

Example:

- *One-tailed test*: "Is the average score greater than 50?"
- *Two-tailed test*: "Is the average score *different* from 50?"

3. Alpha Value (Significance Level)

This is the threshold to decide if a result is significant (commonly 0.05).

- If **p-value ≤ alpha**, we reject the null hypothesis (H_0).
- If **p-value > alpha**, we accept the null hypothesis (H_0).

Statistical Tests in SciPy

1. T-Test (Comparing Two Groups)

The **T-Test** checks if two sets of numbers come from the same group or if they are significantly different.

Example:
Let's check if two random groups have the same average:

```
import numpy as np
from scipy.stats import ttest_ind
# Create two groups of random numbers
v1 = np.random.normal(size=100)
v2 = np.random.normal(size=100)
# Perform T-Test
res = ttest_ind(v1, v2)
print(res)  # Gives t-statistic and p-value
```

If the p-value is **less than 0.05**, the two groups are significantly different.

2. KS-Test (Checking for Normal Distribution)

The **Kolmogorov-Smirnov (KS) Test** checks if a dataset follows a specific distribution (e.g., normal distribution).

Example:
Check if the dataset follows a normal distribution:

```
import numpy as np
from scipy.stats import kstest
v = np.random.normal(size=100)  # Generate random numbers
res = kstest(v, 'norm')  # Test if they follow normal distribution
print(res)  # Gives KS statistic and p-value
```

If p-value is **greater than 0.05**, the data follows a normal distribution.

3. Data Summary (describe())

The describe() function provides an overview of a dataset, including:

- Number of values
- Minimum and maximum values
- Mean (average)
- Variance (spread of data)
- Skewness (symmetry)
- Kurtosis (tail shape)

Example:

```
import numpy as np
from scipy.stats import describe
v = np.random.normal(size=100)  # Generate random data
res = describe(v)
print(res)  # Summary of data
```

4. Normality Tests (Skewness & Kurtosis)

- **Skewness**: Measures if data is symmetrical.
  - **0** = perfectly symmetrical
  - **Negative** = data is skewed to the left
  - **Positive** = data is skewed to the right
- **Kurtosis**: Measures if data has more or fewer extreme values.
  - **Positive** = more extreme values
  - **Negative** = fewer extreme values

Example:

```
import numpy as np
from scipy.stats import skew, kurtosis
v = np.random.normal(size=100)
print(skew(v))  # Skewness of data
print(kurtosis(v))  # Kurtosis of data
```

5. Normality Test (Checking if Data is Normal)

This test checks if a dataset follows a normal distribution.

Example:

```
import numpy as np
from scipy.stats import normaltest
v = np.random.normal(size=100)
print(normaltest(v))  # Gives test statistic and p-value
```

If the **p-value > 0.05**, the data is normally distributed.

Summary

- **T-Test**: Checks if two groups are similar or different.
- **KS-Test**: Checks if data follows a specific distribution.
- **describe()**: Gives a summary of data (mean, variance, etc.).
- **Skewness & Kurtosis**: Checks data symmetry and shape.
- **Normality Test**: Checks if data is normally distributed.

Linear Algebra in SciPy:

What is Linear Algebra?
Linear algebra is a branch of mathematics that deals with **vectors (lists of numbers), matrices (grids of numbers), and solving equations**. It is used in machine learning, physics, engineering, and many other fields.

How Does SciPy Help with Linear Algebra?

SciPy has a module called **scipy.linalg**, which provides functions to perform linear algebra operations easily.

1. Solving Linear Equations

Imagine you have a simple system of equations like this:

2x+3y=82x + 3y = 82x+3y=8 4x+5y=104x + 5y = 104x+5y=10

To find x and y, we can use SciPy.

Code Example

```
import numpy as np
from scipy.linalg import solve
# Coefficients of equations (matrix A)
A = np.array([[2, 3], [4, 5]])
# Constants (matrix B)
B = np.array([8, 10])
# Solve for x and y
solution = solve(A, B)
print(solution)
```

Output:

```
[5.  -2.]
```

So, **x = 5 and y = -2**.

2. Matrix Operations

In linear algebra, matrices are used a lot. You can add, multiply, and find the inverse of matrices using SciPy.

Matrix Multiplication

```
import numpy as np
from scipy.linalg import inv
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
# Multiply matrices
result = np.dot(A, B)
print(result)
```

Output:

```
[[19 22]
 [43 50]]
```

3. Finding the Inverse of a Matrix

The inverse of a matrix is like the "opposite" of a matrix. It helps solve equations easily.

Code Example

```
A = np.array([[1, 2], [3, 4]])
# Find inverse
A_inv = inv(A)
print(A_inv)
```

Output:

```
[[-2.   1. ]
 [ 1.5 -0.5]]
```

4. Eigenvalues and Eigenvectors

What? Eigenvalues and eigenvectors are important in solving systems of equations, quantum mechanics, and data science (e.g., PCA).
How? Use `scipy.linalg.eig()`.

Code Example

```
from scipy.linalg import eig
A = np.array([[3, 1], [2, 4]])
# Find eigenvalues and eigenvectors
eigenvalues, eigenvectors = eig(A)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

Output:

```
Eigenvalues: [2. 5.]
Eigenvectors:
[[-0.707  0.447]
 [ 0.707  0.894]]
```

Summary

- **solve()** → Solves equations like $2x + 3y = 8$
- **np.dot()** → Multiplies matrices
- **inv()** → Finds the inverse of a matrix
- **eig()** → Finds eigenvalues and eigenvectors

Diagonalization in SciPy

What is Diagonalization?
Diagonalization is a process of converting a matrix into a diagonal matrix using **eigenvalues** and **eigenvectors**.

A **diagonal matrix** has non-zero values only on its main diagonal, like this:

$$D = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

where λ_1 **and** λ_2 **are the eigenvalues** of the matrix.

Why is Diagonalization Useful?

- It helps solve matrix equations faster.
- It is used in machine learning, quantum mechanics, and engineering.

How to Diagonalize a Matrix in SciPy

The formula for diagonalization is:

A=PDP−1A = P D P^{-1}A=PDP−1

where:

- **A** = original matrix
- **P** = matrix of eigenvectors
- **D** = diagonal matrix (containing eigenvalues)
- $\mathbf{P^{-1}}$ = inverse of matrix P

Code Example in Python

```
import numpy as np
from scipy.linalg import eig, inv
# Define a matrix A
A = np.array([[4, 2],
              [1, 3]])
# Step 1: Find eigenvalues and eigenvectors
eigenvalues, P = eig(A)
# Step 2: Create diagonal matrix D using eigenvalues
D = np.diag(eigenvalues)
# Step 3: Find inverse of P
P_inv = inv(P)
# Verify: A = P D P⁻¹
A_reconstructed = P @ D @ P_inv
print("Matrix A:")
print(A)
print("\nEigenvalues (Diagonal Matrix D):")
print(D)
print("\nEigenvectors (Matrix P):")
print(P)
print("\nReconstructed A (Should be same as original A):")
print(A_reconstructed)
```

Explanation

1. **Find eigenvalues and eigenvectors** using eig(A).
2. **Create a diagonal matrix D** using the eigenvalues.
3. **Find the inverse of the eigenvector matrix P**.
4. **Check if A = P D P^{-1}**, meaning the diagonalization is correct.

Example Output

```
Matrix A:
[[4 2]
 [1 3]]
Eigenvalues (Diagonal Matrix D):
[[5. 0.]
 [0. 2.]]
Eigenvectors (Matrix P):
[[ 0.894  0.707]
 [ 0.447 -0.707]]
Reconstructed A (Should be same as original A):
[[4. 2.]
 [1. 3.]]
```

Key Points

Diagonalization helps simplify matrix calculations.
Eigenvalues go into the diagonal matrix D.
Eigenvectors form matrix P, and its inverse helps reconstruct A.

Matrix Factorizations

These break down matrices into simpler forms, useful in optimization and signal processing.

LU Decomposition

What? Factorizes a matrix into **Lower (L) and Upper (U) matrices**.
How? Use scipy.linalg.lu().

Example:

```
from scipy.linalg import lu
A = np.array([[4, 3], [6, 3]])
P, L, U = lu(A)
print("Lower Matrix L:\n", L)
print("Upper Matrix U:\n", U)
```

3.2 QR Decomposition

What? Factorizes A into **Q (orthogonal) and R (upper triangular) matrices**.
How? Use scipy.linalg.qr().

Example:

```
from scipy.linalg import qr
A = np.array([[1, 1], [1, -1], [1, 2]])
Q, R = qr(A)
print("Q Matrix:\n", Q)
print("R Matrix:\n", R)
```

Singular Value Decomposition (SVD)

What? Breaks a matrix into three matrices: **U, Σ, V**.
How? Use scipy.linalg.svd().

Example:

```
from scipy.linalg import svd
A = np.array([[1, 2], [3, 4], [5, 6]])
U, S, Vh = svd(A)
print("U Matrix:\n", U)
print("Singular Values:\n", S)
print("V Transpose Matrix:\n", Vh)
```

Determinants and Rank

What? Measures whether a matrix is invertible.
How? Use scipy.linalg.det().

Example:

```
from scipy.linalg import det
A = np.array([[2, 3], [1, 4]])
print("Determinant:", det(A))
```

Finding Rank

What? Tells us how many independent rows/columns a matrix has.
How? Use numpy.linalg.matrix_rank().

Example:

```
from numpy.linalg import matrix_rank
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Rank:", matrix_rank(A)
```

Norms (Matrix and Vector)

What? Measures the "size" of a vector or matrix.
How? Use scipy.linalg.norm().

Example:

```
from scipy.linalg import norm
A = np.array([[3, 4], [5, 12]])
print("Matrix Norm:", norm(A))  # Frobenius norm by default
```

Integration Using SciPy:

Integration is the process of finding the area under a curve. In SciPy, we can use the scipy.integrate module to perform different types of integrations, including **definite integrals, indefinite integrals, and solving differential equations**.

Single Integration (Definite Integral)

What? Finds the area under a curve between two points **(a to b)**.
How? Use scipy.integrate.quad().

Example: Compute the integral of f(x) = x^2 from 0 to 2.

```
from scipy.integrate import quad
# Define function
def f(x):
    return x**2
# Compute integral from 0 to 2
result, error = quad(f, 0, 2)
print("Integral result:", result)
print("Estimated error:", error)
```

Output:

```
Integral result: 2.666666666666667
Estimated error: 2.960594732333751e-14
```

Explanation: This finds the area under x^2 from 0 to 2.

Double Integration (2D Integral)

What? Computes integrals of functions with two variables.
How? Use scipy.integrate.dblquad().

Example: Compute the double integral of f(x, y) = x * y for x: 0 to 1, y: 0 to 2

```
from scipy.integrate import dblquad
# Define function
def f(x, y):
    return x * y
# Compute double integral
result, error = dblquad(f, 0, 1, lambda x: 0, lambda x: 2)
print("Double Integral result:", result)
```

Output: Double Integral result: 1.0 **Explanation:** This computes the volume under x * y over the given range.

Triple Integration (3D Integral)

What? Computes integrals for functions with three variables.
How? Use scipy.integrate.tplquad().

Example: Compute the triple integral of $f(x, y, z) = x + y + z$ for

- **x: 0 to 1**
- **y: 0 to 1**
- **z: 0 to 1**

```
from scipy.integrate import tplquad
# Define function
def f(x, y, z):
    return x + y + z
# Compute triple integral
result, error = tplquad(f, 0, 1, lambda x: 0, lambda x: 1, lambda x, y: 0, lambda x, y: 1)
print("Triple Integral result:", result)
```

Output:

```
Triple Integral result: 1.5
```

Explanation: This computes a 3D volume integral.

Indefinite Integral (Symbolic Integration)

What? Computes indefinite integrals (antiderivatives).
How? Use **SymPy** (since SciPy does not support symbolic integration).

Example: Compute $\int(x^2)dx$

```
from sympy import symbols, integrate
x = symbols('x')
f = x**2
# Compute indefinite integral
indef_integral = integrate(f, x)
print("Indefinite Integral:", indef_integral)
```

Output:

```
Indefinite Integral: x**3/3
```

Explanation: This gives $\mathbf{x^3/3}$, the antiderivative of x^2.

Solving Differential Equations

What? Solves equations like $dy/dx = f(x, y)$.
How? Use scipy.integrate.odeint().

Example: Solve $dy/dx = -y$ with $y(0) = 1$

```
from scipy.integrate import odeint
import numpy as np
# Define function dy/dx = -y
def model(y, x):
    return -y
x = np.linspace(0, 5, 100)  # Time points
y0 = 1  # Initial condition
# Solve ODE
y = odeint(model, y0, x)
# Print solution at x = 5
print("Solution at x=5:", y[-1])
```

Explanation: This solves the differential equation and returns values of y at different x.

Summary:

✓ **quad()** → Single integral
✓ **dblquad()** → Double integral
✓ **tplquad()** → Triple integral
✓ **integrate() (SymPy)** → Indefinite integral
✓ **odeint()** → Solves differential equations

What is MATLAB?

MATLAB (MATrix LABoratory) is a programming language and environment used for **mathematical computing, data analysis, and visualization**. It is widely used in engineering, science, and machine learning.

Key Features of MATLAB:

1. **Matrix-Based Computation:** MATLAB is designed to work with matrices and arrays efficiently.
2. **Data Visualization:** It provides built-in functions for plotting graphs, 3D visualizations, and image processing.
3. **Built-in Functions & Toolboxes:** MATLAB has many specialized toolboxes for machine learning, image processing, signal processing, and more.
4. **Simulations & Modeling:** Engineers use MATLAB to simulate systems, such as control systems and neural networks.

Example of MATLAB Code:

```
A = [1 2 3; 4 5 6; 7 8 9];  % Create a 3x3 matrix
B = A * 2;  % Multiply all elements by 2
disp(B);  % Display the result
```

Output:

```
 2   4   6
 8  10  12
14  16  18
```

This example creates a matrix `A`, multiplies it by `2`, and displays the result.

MATLAB vs. Python (NumPy & SciPy)

- **MATLAB** is a paid software mainly used in academia and research.
- **Python** (with **NumPy, SciPy, and Matplotlib**) is **free** and widely used for similar tasks.
- **SciPy's `scipy.io` module** helps in **saving and loading MATLAB `.mat` files** in Python.

SciPy and MATLAB Arrays – Simple Explanation for Beginners

SciPy provides tools to work with **MATLAB arrays** using the `scipy.io` module. This means you can **save and load** data in MATLAB format (`.mat` files) using Python.

Saving Data in MATLAB Format

To save data in MATLAB format, we use the **`savemat()`** function.

How it Works:

- **Filename:** The name of the file where data will be saved.
- **Dictionary (mdict)**: A Python dictionary where **keys are variable names** and **values are the data**.
- **Compression (do_compression)**: Whether to compress the file (default is False).

Example: Save an array to MATLAB format

```
from scipy import io
import numpy as np
# Create a NumPy array
arr = np.arange(10)  # [0,1,2,3,4,5,6,7,8,9]
# Save the array in a .mat file with variable name "vec"
io.savemat('arr.mat', {"vec": arr})
```

This creates a **file named arr.mat** on your computer, which contains the array stored as "vec".

Loading Data from a MATLAB File

To read data from a .mat file, we use the **loadmat()** function.

Example: Load the saved MATLAB file

```
# Load the .mat file
mydata = io.loadmat('arr.mat')
# Print the entire loaded data
print(mydata)
```

The result will be a **dictionary** containing metadata and the saved array:

```
{
  '__header__': b'MATLAB 5.0 MAT-file ...',
  '__version__': '1.0',
  '__globals__': [],
  'vec': array([[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]])
}
```

Accessing Only the Array from the MATLAB File

To extract only the saved array, use its variable name (e.g., "vec"):

```
print(mydata['vec'])
```

Output:

```
[[0 1 2 3 4 5 6 7 8 9]]
```

Notice how the array is **wrapped in extra brackets** ([[...]]), making it **2D** instead of 1D.

Fixing Extra Dimensions in the Loaded Array

To get the array in its original **1D form**, add squeeze_me=True when loading:

```
mydata = io.loadmat('arr.mat', squeeze_me=True)
print(mydata['vec'])
```

Output:

```
[0 1 2 3 4 5 6 7 8 9]
```

Now, the array is **1D**, just as we originally created it.

Summary

1. **Save Data** to a MATLAB file using io.savemat('filename.mat', {"variable_name": array}).

   Load Data from a MATLAB file using io.loadmat('filename.mat').

2. Extract a **specific variable** using its name (mydata['vec']).
3. Use **squeeze_me=True** to remove unwanted dimensions.